



CAHIER DES CHARGES TECHNIQUE DU PROJET SYNAFIG DIGITAL

Nom du projet : SYNAFIG DIGITAL

Date : 11 Août 2026

Nom des développeurs : Kouassi Yao Jean Paul Danick,

Adresse GitHub du projet : <https://github.com/jeanpauldeveloper24/SYNAFIG-digital-project>

1. Analyse client

Le **SYNAFIG** (Syndicat National des Agents des Finances Générales de Côte d'Ivoire) est l'organisation syndicale regroupant les agents publics des finances répartis sur l'ensemble du territoire national ivoirien.

Le projet **SYNAFIG DIGITAL** s'inscrit directement dans le cadre de la campagne électorale pour l'élection du nouveau Secrétariat Général du syndicat. Parmi les différentes candidatures en compétition, l'une des candidates positionne la **transformation numérique** comme l'un des piliers majeurs de son programme d'actions.

Elle s'engage, en cas d'élection, à doter le SYNAFIG d'un écosystème numérique complet (un site web institutionnel/portail d'administration et une application mobile pour les membres). L'objectif de cette proposition est de démontrer par des actes concrets une volonté de **moderniser l'institution**, d'assurer une **transparence financière et administrative**, de **renforcer la proximité** avec les agents (particulièrement ceux basés à l'intérieur du pays) et d'optimiser la réactivité des services syndicaux.

2. Contexte du projet

Le présent document constitue une **proposition technique et fonctionnelle proactive** élaborée par l'équipe de développement. Sans ateliers de cadrage préalables avec la candidate ou le bureau sortant, cette analyse s'appuie sur une observation générale du fonctionnement des organisations syndicales de la fonction publique en Côte d'Ivoire.

L'objectif de cette proposition est d'anticiper et d'adresser les défis opérationnels classiques auxquels le **SYNAFIG** est susceptible de faire face :

- **Centralisation géographique** : Une forte dépendance vis-à-vis des bureaux d'Abidjan pour l'accès aux informations officielles et l'effectuation des démarches administratives.
- **Gestion manuelle des adhésions** : Les risques de perte d'informations ou de lenteur liés à la tenue de registres physiques pour le suivi des membres.
- **Traçabilité des cotisations** : La complexité d'un suivi financier manuel sans outil d'encaissement et de consolidation automatisé.
- **Communication et recours** : Des délais importants dans la diffusion des communiqués et l'acheminement des demandes d'assistance des agents.

Le projet **SYNAFIG DIGITAL** est donc pensé comme un **modèle de solution clé en main** à lui présenter : une architecture combinant un **portail Web d'administration** et une **application Mobile multiplateforme** visant à démontrer la faisabilité et la valeur ajoutée de sa promesse de campagne.

3. Contraintes techniques

- **Haute disponibilité et performance** : Temps de réponse fluide (< 2s) même en cas d'affluence lors des convocations d'Assemblées Générales.
- **Compatibilité réseau (2G/3G/4G)** : L'application mobile doit continuer de fonctionner partiellement en mode déconnecté (consultation de la carte de membre et cache des actualités).
- **Sécurité & Législation** : Conformité stricte aux directives de l'**ARTCI** concernant le stockage et le traitement des données à caractère personnel des agents de l'État.
- **Pérennité** : Architecture dimensionnée pour supporter une montée en charge sur plus de 10 ans sans refonte lourde.

4. Choix technologiques pour le front-end

- **Web (Portail Admin & Adhérent) :**
 - **Blade (Laravel)** : Moteur de templating natif pour un rendu rapide côté serveur, facilitant l'intégration des vues.
- **Mobile (Application Adhérent) :**
 - **Flutter (Dart)** : Développement multiplateforme (iOS & Android) offrant des performances natives, une grande réactivité d'affichage et une gestion simplifiée de l'état avec flutter_bloc ou provider.

5. Choix technologiques pour le back-end

- **Framework Backend : Laravel (PHP)**
 - **Gestion unifiée** : Sert à la fois le back-office web et l'API RESTful pour le mobile.
 - **Laravel Breeze** : Gestion sécurisée de l'authentification web (connexion, réinitialisation de mot de passe, sessions).

- **Laravel Sanctum** : Génération et validation des jetons de sécurité (*Personal Access Tokens*) pour l'application Flutter.
- **Base de Données : MySQL (Moteur InnoDB)**
 - Respect strict des transactions ACID pour sécuriser la gestion des cotisations et reçus financiers.
 - Indexation B-Tree optimisée pour la recherche rapide sur les matricules et identifiants.

6. L'architecture technique

L'écosystème repose sur une **architecture client-serveur découplée via API REST** :

Plaintext

[Application Mobile Flutter] <-- (API REST / HTTPS JSON) --> [Backend Laravel] <--> [MySQL Database]

[Portail Web Admin Blade] <-- (SSR / Blade Templates) --> [Backend Laravel] <--> [MySQL Database]

1. **Couche Présentation** : Vues Blade côté Web et composants Widget côté Mobile.
2. **Couche Service / API** : Contrôleurs Laravel traitant les requêtes, appliquant la logique métier et renvoyant du JSON (pour l'API) ou des vues HTML (pour le web).
3. **Couche Données** : ORM Eloquent assurant la liaison sécurisée avec MySQL.
4. **Services Tiers** :
 - **Firebase Cloud Messaging (FCM)** : Pour l'envoi de notifications push ciblées.
 - **Agrégateur Mobile Money (Passerelle API)** : Pour le règlement des cotisations (Wave, Orange, MTN, Moov).

7. Stratégie de sécurité

- **Chiffrement des données** : HTTPS/TLS sur l'ensemble des échanges Web et Mobile. Hachage fort des mots de passe via l'algorithme **Bcrypt**.
- **Protection contre les vulnérabilités OWASP** :
 - Injections SQL évitées grâce au Binding d'ORM Eloquent.
 - Failles CSRF contrecarrées par les jetons CSRF natifs de Laravel.
 - Failles XSS évitées par l'échappement automatique du moteur Blade.

- **Gestion fine des permissions** : Attribution de rôles stricts via spatie/laravel-permission (ADHERENT, DELEGUE_REGIONAL, ADMIN_BUREAU, SUPER_ADMIN).

8. Déploiement

- **Environnement de Pré-production** : Serveur VPS Linux (Ubuntu 24.04 LTS) configuré avec Nginx, PHP-FPM et MySQL.
- **CI/CD** : Actions GitHub pour l'exécution automatique des tests unitaires et le déploiement sur le serveur.
- **Mobile** : Génération des livrables .apk (Android) et déploiement sur Google Play Store & Apple App Store.

9. Version web

Prérequis

- PHP >= 8.2 (extensions : mbstring, pdo, pdo_mysql, openssl, tokenizer, xml)
- Composer >= 2.x
- Node.js >= 18.x & NPM
- Serveur de base de données MySQL >= 8.0

Installation

1. **Cloner le dépôt Git** : pour plus de rapidité nous utiliserons github desktop
2. **Installer les dépendances PHP et JavaScript** :

composer install

npm install

3. **Configurer l'environnement** :

cp .env.example .env

php artisan key:generate

Renseigner les accès MySQL (DB_DATABASE, DB_USERNAME, DB_PASSWORD) dans le fichier .env.

4. Exécuter les migrations et compiler les assets :

php artisan migrate --seed

npm run build

5. Lancer le serveur local :

php artisan serve

10. Méthode de travail

Projet géré selon la méthodologie **Agile (Scrum)** découpée en itérations d'une semaine (Sprints) :

- **Sprint 1** : Modélisation de la BDD, configuration Laravel/Breeze/Sanctum et maquettage UI/UX.
- **Sprint 2** : Développement des modules d'authentification et gestion du profil utilisateur (Web/Mobile).
- **Sprint 3** : Développement du module d'adhésion, génération du QR Code membre et module de cotisations.
- **Sprint 4** : Intégration du fil d'actualités, des tickets d'assistance et du système de notifications Push.
- **Sprint 5** : Recettage, tests d'intégration, recette utilisateur et déploiement.

11. Outils utilisés

- **Environnements de développement (IDE)** : VS Code.
- **Gestion de versions & Collaboration** : GitHub.
- **Conception BDD & Diagrammes** : MySQL Workbench.
- **Test des API** : Postman.
- **Maquettage & Design** : Figma.

12. Évaluation du temps de travail

Poste de développement	Temps estimé	Détails
Backend (Laravel)	40 heures	Modélisation BDD, configuration du projet, API Sanctum, logique métier cotisations/membres.
Développeur Web Frontend (Blade)	30 heures	Intégration de la charte graphique, création des vues d'administration et formulaires d'adhésion.
Développeur Mobile (Flutter)	45 heures	Intégration des écrans, consommation des API, génération du QR Code, gestion SQLite hors-ligne.
Recette, Tests & Déploiement	15 heures	Tests de sécurité, tests de montée en charge, déploiement VPS et configuration CI/CD.
TOTAL	130 heures	Durée estimée globale du projet.

13. Liste fonctionnelle

Fonctionnalité	Front-End	Back-End
Authentification & Profil	Écrans Login/Register (Web Blade & Mobile Flutter) avec validation.	Contrôleurs Laravel, génération de Tokens Sanctum, hachage Bcrypt.
Carte de Membre Numérique	Affichage du QR Code dynamique et détails du membre dans Flutter.	Endpoint API de vérification du profil et statut de validité.
Gestion des Cotisations	Interface de choix du montant/période et déclenchement Mobile Money.	Webhook de confirmation de paiement, enregistrement transactionnel MySQL.
Actualités & Communiqués	Liste filtrable d'articles (Web & Mobile) avec mise en cache.	CRUD des articles dans le Dashboard Admin, diffusion via API REST.
Assistance / Tickets	Formulaire de soumission de litige et suivi du statut du ticket.	Module de gestion des tickets côté bureau, notifications par email/push.
Notifications Push	Réception des alertes instantanées sur smartphone via FCM.	Intégration du SDK Firebase pour le déclenchement des envois groupés.

14. Recettage

Fonctionnalité	État
Connexion / Déconnexion Web & Mobile	
Inscription d'un nouvel adhérent avec upload de justificatif	
Génération du QR Code de carte membre	
Paieement d'une cotisation via Mobile Money (Sandbox)	
Envoi et réception d'une notification Push	
Création et modération d'une actualité depuis l'Admin	

15. Diagramme ERP de la base de données

Relations entre les tables :

- users **(1)** ---> **(N)** cotisations : Un utilisateur peut réaliser plusieurs cotisations.
- users **(1)** ---> **(N)** tickets : Un utilisateur peut ouvrir plusieurs demandes d'assistance.
- users **(1)** ---> **(N)** actualites : Un administrateur peut rédiger plusieurs articles.

SQL

```
CREATE DATABASE IF NOT EXISTS synafig_db CHARACTER SET utf8mb4 COLLATE  
utf8mb4_unicode_ci;
```

```
USE synafig_db;
```

```
CREATE TABLE users (  
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
    matricule VARCHAR(50) UNIQUE NOT NULL,  
    nom VARCHAR(100) NOT NULL,  
    prenom VARCHAR(100) NOT NULL,  
    email VARCHAR(150) UNIQUE NOT NULL,  
    telephone VARCHAR(20) NOT NULL,
```



```

password VARCHAR(255) NOT NULL,
grade VARCHAR(100) NULL,
direction VARCHAR(150) NULL,
region_administrative VARCHAR(100) NULL,
role ENUM('ADHERENT', 'DELEGUE', 'ADMIN') DEFAULT 'ADHERENT',
statut_adhesion ENUM('EN_ATTENTE', 'VALIDE', 'SUSPENDU', 'REJETE') DEFAULT
'EN_ATTENTE',
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
) ENGINE=InnoDB;

```

```

CREATE TABLE cotisations (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT UNSIGNED NOT NULL,
    montant DECIMAL(10, 2) NOT NULL,
    mois INT NOT NULL,
    annee INT NOT NULL,
    moyen_paiement ENUM('WAVE', 'ORANGE_MONEY', 'MTN_MOMO', 'MOOV_MONEY')
NOT NULL,
    reference_transaction VARCHAR(100) UNIQUE NOT NULL,
    statut ENUM('EN_ATTENTE', 'SUCCES', 'ECHEC') DEFAULT 'EN_ATTENTE',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB;

```

```

CREATE TABLE actualites (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    titre VARCHAR(255) NOT NULL,

```

```
slug VARCHAR(255) UNIQUE NOT NULL,  
contenu TEXT NOT NULL,  
image_url VARCHAR(255) NULL,  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
) ENGINE=InnoDB;
```

```
CREATE TABLE tickets (  
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
    user_id BIGINT UNSIGNED NOT NULL,  
    sujet VARCHAR(200) NOT NULL,  
    description TEXT NOT NULL,  
    priorite ENUM('BASSE', 'MOYENNE', 'HAUTE') DEFAULT 'MOYENNE',  
    statut ENUM('OUVERT', 'EN_COURS', 'RESOLU') DEFAULT 'OUVERT',  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE  
) ENGINE=InnoDB;
```

16. Wireframes

Les wireframes détaillent la disposition des éléments clés du projet :

- **Web Admin** : Menu latéral fixe, barre de navigation supérieure avec profil, zone centrale d'affichage des statistiques (nombre d'adhérents, montant total des cotisations) et tableaux de données paginés.
- **Mobile Flutter** :
 - *Écran d'accueil* : Bannière actualités, statut du membre, accès rapide aux cotisations.
 - *Carte Membre* : Vue épurée contenant la photo, le nom, le matricule et le QR code scannable.

17. Le journal de développement

Comptes Rendus de Développement & Difficultés rencontrées :

Comptes Rendus des Tests Utilisateurs :

Testeurs : Équipe de développement et échantillon de 5 agents testeurs du SYNAFIG.